APPENDIX [X]

The Specification Dialogue: How the Agent Developed the Requirements

Purpose and scope

The main body of this application note documents the autonomous phase of the demonstration: an 18-minute unattended session in which the agent wrote, compiled, flashed, and validated the RO skid controller on a live EQSP32, with 31 of 31 checks passing. This appendix documents what preceded it — the specification dialogue in which the requirements themselves were developed.

It is included for two reasons. First, it shows the complete workflow honestly: the demonstration was not a single prompt producing firmware, but a two-phase process — a collaborative specification phase in which the engineer supplied documents and decisions and the agent performed requirements engineering, followed by an autonomous implementation and validation phase executed against the approved specification. Second, the dialogue itself is evidence of a kind that test results cannot provide: it shows the agent exercising engineering judgment — validating its inputs, disclosing its assumptions, catching a physically unimplementable requirement, and submitting its own design additions for human confirmation.

1 First turn — a functional specification from a wiring diagram

> **USER (SUMMARIZED)**
>
> *The engineer attached what he believed were two documents — the wiring diagram and the system diagram — and asked for a specification of how the skid will operate, supplying the requirements core directly in the prompt: run only while the supply tank holds 50 cm or more, and until the clean-water tank reaches 400 cm; flush the membrane for 5 minutes after every 60 minutes of operation; motor pump temperature below 80 °C; supply flow around 10 L with permeate between 10% and 30% of it; TDS below 200; membrane pressures in and out between 150 and 200 PSI; "if pressure IN is above 200 while flow out is low, this means the membrane needs cleaning"; and a full RGB signaling scheme — green running, yellow flushing, blue stopped on full tank, yellow at different flash rates for marginal conditions, red for error stops. He closed by asking the agent to flag any other points that needed specifying.*

The agent's first action was not to specify anything, but to validate its inputs:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> I've reviewed both attachments. Note: both saved files are actually
> the same wiring diagram — I didn't receive a separate water-
> circuit/process (P&ID) drawing, so a couple of plumbing assumptions
> below are flagged for you to confirm.
> ```

Annotation — *The engineer had intended to attach two different documents — the wiring diagram and the system diagram — but both files were the same drawing; no process diagram had actually been supplied. Rather than inventing a plausible plumbing topology, the agent declared the gap, proceeded on explicitly labeled assumptions, and requested the missing document. Input validation before work is precisely the behavior the term "AI slop" presumes to be absent.*

The draft specification that followed contained a complete I/O assignment read from the wiring diagram, an operating-mode/state model, a full sequence of operation with proposed numeric setpoints (each marked as a proposal to confirm), a protection table with debounce recommendations, and a prioritized RGB indication scheme. Two passages deserve particular attention. The first is an assumption flagged together with its downstream consequence:

> AGENT (VERBATIM EXCERPT)
>
> Assumption to confirm: pump is fixed-speed (relay/contactor) and operating pressure is set mechanically; the “flush valve” is the concentrate/forward-flush valve (open = low pressure high crossflow, closed = build pressure / produce). There is no permeate divert-to-drain valve in the wiring (this matters for startup water quality — see §10).

Annotation — *The agent noticed an actuator that was not there and reasoned about what its absence implies: the first permeate after startup is typically high-TDS, and without a divert valve the only available response to out-of-spec water is a stop. It later posed this as an explicit design decision — add a fourth output, or accept stop-on-high-TDS. The human chose the latter, and that decision is traceable through every subsequent revision.*

The second is the closing section of the draft — a list of fourteen open engineering points. A representative sample:

> AGENT (VERBATIM EXCERPT)
>
> Process/P&ID diagram — both attached files are the same wiring diagram; a water-circuit drawing would confirm the flush valve's plumbing, concentrate path and permeate path.
>
> No permeate divert valve — first permeate after start is usually high-TDS. Without a divert-to-drain valve we can only stop on high TDS, not divert. Add a 4th output for a permeate divert valve, or accept “stop on high TDS”?
>
> Tank hysteresis / restart levels for both tanks (proposed: clean restart 380 cm, supply resume 60 cm).
>
> Spare ADIO8 — strongly suggest using it for an E-stop or leak/flood sensor; is local emergency stop required?
>
> Pre-treatment (cartridge filters) and pump suction — is feed gravity-fed at adequate pressure to avoid pump cavitation, or is a feed/booster pump present?
>
> Temperature compensation of permeate flow/TDS (RO flux changes ~3%/°C) — include normalization or keep raw thresholds?

Annotation — *The engineer explicitly invited this list — his prompt closed with “let me know if you see any other points that we need to specify.” What the invitation could not supply is the content, and each question carries its engineering rationale: hysteresis to prevent short-cycling, the spare channel proposed for safety functions, cavitation risk tied to the suction arrangement, temperature compensation tied to the physics of membrane flux. Throughout the draft the agent also committed to concrete default values — anti-short-cycle timing, settling windows, debounce times — rather than leaving blanks, while marking every number as a proposal awaiting confirmation. Propose-then-confirm keeps the human in the design loop without stalling the work.*

2 Second turn — revision against the real P&ID and the engineer's decisions

> **USER (SUMMARIZED)**
>
> *The engineer resupplied the corrected diagrams ("I sent you the wrong diagram — here they are again") and reported a hardware change made in the interim: a Run/Stop button added to the skid, to operate as an emergency stop, with STOP also clearing faults. He then disposed of the open points in one pass: no permeate flush at startup and none after power-up; MQTT, telemetry and other cloud features out of scope for now; water is gravity-fed with no need for pumping; no temperature compensation; no PID — the pump is simple on/off; tank hysteresis approved; minimum on and minimum off times of 10 seconds; the system runs immediately upon power-on if the button is on RUN; the NTC is a 10K part with default characteristics. He asked for the specification to be updated and saved as a Word file.*

The agent returned a revised specification with a change log in which every modification is tied either to a human decision or to information from the corrected P&ID — sensor scaling filled in from the drawing, flush mechanics clarified from the actual plumbing, and the excluded features moved to an explicit out-of-scope section rather than silently dropped. One detail of this exchange deserves its own note: the Run/Stop button was a physical addition the engineer made to the skid between turns, following the agent's open-point suggestion that an emergency-stop function be considered — the one moment in the dialogue where an agent question changed the hardware, not just the firmware. The agent then raised two items under the heading "Two things worth your attention." The first is the standout moment of the entire dialogue:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> Pressure transmitters are 0–200 PSI (20 mA = 200 PSI), so they
> physically can't read above 200. I reinterpreted your fouling rule
> as "feed pressure at/near full-scale (≈200) + low permeate flow." If
> you want a true over-200 PSI trip, you'd need a 0–250/300 PSI
> transmitter. (Flagged in the doc.)
> ```

Annotation — *The engineer's membrane-fouling rule — carried verbatim from his first prompt — referenced pressures above 200 PSI; the specified instruments saturate at exactly 200 PSI and can never report such a value. The requirement as written was unimplementable — not wrong in logic, but wrong against the physics of the installed sensor. The agent caught the conflict, proposed a faithful reinterpretation within the hardware's limits, offered the hardware change that would satisfy the original intent, and flagged the issue in the document for a human decision. This is instrumentation-engineering reasoning, and it is the kind of error that survives code review, compiles cleanly, and passes any test written against the same mistaken assumption.*

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> No permeate divert valve exists in the wiring, so out-of-spec
> permeate (TDS > 200) can only stop the skid, not divert to drain.
> ```

Annotation — *The divert-valve limitation identified in the first turn is restated at revision time rather than being absorbed silently — the constraint remains visible at every stage at which the human could still change the design.*

3 Third turn — the completion report of the autonomous run

> **USER (SUMMARIZED)**

> *The engineer confirmed the transmitter catch ("You are right about the pressure sensor") and approved extending the range to 300 PSI as the agent had suggested; set the permeate rule to stop the skid if TDS stays above 200 for 10 minutes; and then issued the implementation instruction: write and run the code for the EQSP32, still connected on port 35; write and run a validation test program using the HIL capabilities; produce a document explaining what is tested and how; and produce a test report when all tests pass.*

What followed was the autonomous session documented in the main body of this note. The agent's completion report opens:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> Done. Everything is built, flashed, validated on the live EQSP32
> (COM35), and documented — 31/31 checks pass.
> ```

The report enumerates the deliverables — the updated specification (v0.3), the firmware implementing the full state machine (STOP / IDLE_FULL / PAUSE_SUPPLY / STARTING / RUNNING / FLUSHING / FAULT), the HIL test harness, and the generated test plan and test report documents. Beyond the headline result, three passages matter for assessing the credibility of the validation. The first discloses the mechanics of the test harness:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> Two engineering notes worth flagging: flow uses pulse-count (PCC)
> inputs that are consumed on read, so the harness re-applies the full
> input vector as one periodic frame to keep flows fed; and long
> timers are time-compressed in the HIL build (logic/setpoints
> identical to production) so the whole thing runs in ~80 s.
> ```

Annotation — *The agent volunteers exactly the two places where the HIL run differs from production reality — the periodic re-injection required by consumed-on-read pulse counters, and the time compression of long process timers — and states what is preserved (logic and setpoints identical). A validation whose mechanics are disclosed can be audited; one that reports only a green banner cannot. Note also that this disclosure is unprompted.*

The second concerns the state in which the physical hardware was left, and the path from HIL build to production build:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> The device was left safely stopped (Run/Stop = STOP) at the end of
> the run.
> (a) the firmware has #define HIL_TEST at the top — comment it out to
> restore the real 10 s / 60 min / 5 min / 10 min timers and remove
> HIL injection;
> ```

Annotation — *Safe-state discipline at the end of an unattended session on live hardware, and a single, explicit, documented switch separating the test build from the production build.*

The third is a disclosure of the agent's own design contribution:

> **AGENT (VERBATIM EXCERPT)**
>
> ```
> (b) the 250 PSI over-pressure trip is my addition enabled by the 300
> PSI range — it's listed as an open point in the spec for your
> confirmation.
> ```

Annotation — *The agent added a protection function that the human never requested — made possible by the newly extended transmitter range — implemented it, tested it, and then explicitly*

labeled it as agent-originated and pending human confirmation. Design authority remained with the engineer even for an improvement the agent was confident enough to build.

4 What the dialogue demonstrates

Read as a whole, the transcript documents four behaviors that together answer the standing objection to AI-generated control code. First, input validation and assumption disclosure: the agent checked what it had actually been given, worked on labeled assumptions where information was missing, and requested the missing artifact. Second, domain reasoning: the transmitter-range catch is not a property of language modeling in the abstract but of applying instrument physics to a stated requirement — the requirement was rejected not because it was illogical but because the installed sensor could never produce the triggering value. Third, a consistent propose-then-confirm pattern: concrete defaults were always offered, never silently imposed, and the one design decision the agent originated on its own — the 250 PSI trip — was flagged for sign-off even after it had been implemented and tested. Fourth, the two-phase structure of the work itself: the human's contribution moved up the abstraction ladder, from writing and debugging code to supplying documents, making design decisions, and approving requirements — after which implementation and proof were delegated entirely.

The 31/31 result in the main body of this note establishes that the delivered firmware meets its specification. This appendix establishes something the test matrix cannot: that the specification itself was developed with the skepticism, traceability, and disclosure that professional controls engineering demands — and that the agent, not only the human, enforced those standards.